

NovaCraft

A Statically-Typed Compiler to WebAssembly with
Provably-Eliminated Array Bounds Checks

Project Review — Technical Report
Prepared by Harshit Chhabi
September 17, 2026

This report documents the design, implementation, and validation of NovaCraft: a from-scratch compiler for a small imperative language that lexes, parses, type-checks, optimizes, register-allocates, and code-generates all the way down to real, executable WebAssembly text — with a working range-analysis pass that proves array accesses safe at compile time and removes their runtime checks.

Executive Summary

NovaCraft is a complete, working compiler — not a toy parser or an interpreter — for a small statically-typed imperative language. Every classical compiler stage is genuinely implemented and independently inspectable: a hand-written lexer and recursive-descent parser, a scoped semantic analyzer, a structured three-address intermediate representation (IR), four real optimization passes, a linear-scan register allocator with spilling, and a code generator that emits real WebAssembly text, assembled and executed via Node's built-in **WebAssembly** engine.

The project's defining feature is memory safety without a fixed performance tax: every array access in the language is bounds-checked by construction, but a compile-time range-analysis pass proves a large class of those checks redundant and deletes them before code generation, rather than either trusting the programmer (C/C++'s approach) or paying for every check at runtime unconditionally (a naive safe-language approach). A dedicated benchmark demonstrates this is not just a proof obligation but a real, measured performance win: 33%–75% of the per-access overhead removed in the two measured scenarios.

Beyond the language and compiler itself, the project includes a genuine downstream application — a moving-average anomaly detector for numeric time series — built on top of the compiler as a real consumer, not a demo. Building that application surfaced and led to the fix of an actual, previously-undetected bug in the range-analysis pass, which this report documents as a case study in Section 8. The project carries 34 automated tests across 10 suites, a continuous-integration workflow, and a documented design log capturing the reasoning behind every non-obvious decision.

Contents

1. Motivation & Problem Statement
2. Language Design
3. Compiler Architecture & Pipeline
4. Deep Dive: Bounds-Check Elimination
5. Deep Dive: Register Allocation & Calling Convention
6. Runtime Safety: Traps and Source Maps
7. Engineering Additions: Control-Flow Sugar, CSE, and a Codegen Fix
8. Case Study: A Real Application and the Bug It Found
9. Testing Strategy & Continuous Integration
10. Performance Evaluation
11. Limitations & Future Work
12. Conclusion

1. Motivation & Problem Statement

Out-of-bounds array access is one of the oldest and most consequential classes of software defects: it underlies a large fraction of historical memory-safety vulnerabilities and, even when not exploited maliciously, is a routine source of crashes and data corruption. Languages resolve this tension between safety and performance in one of two ways. Systems languages without automatic bounds checking (C, and unchecked code in C++) leave the responsibility entirely to the programmer, trading safety for raw speed. Managed or "safe" languages (Java, Python, JavaScript's typed arrays) check every single array access at runtime unconditionally, trading a predictable, universal performance cost for guaranteed safety.

NovaCraft explores the third option that ahead-of-time optimizing compilers actually use in production systems (this is the same family of technique behind, for example, the JVM's and V8's bounds-check-elimination passes): keep every access checked by default for safety, but let a static analysis *prove* that a specific check can never fail, and delete only that check. The project's goal was to implement this end to end and honestly — not as a hard-coded special case for one example program, but as a real, general data-flow analysis that correctly identifies both when a check can be removed and, just as importantly, when it genuinely cannot.

WebAssembly was chosen as the compilation target for two reasons. First, it is a real, production-grade instruction set with a structured control-flow model, a linear memory model, and an actual runtime (available for free via Node.js) to execute the generated code against — so "the compiler works" can be verified by actually running the output, not just by inspecting it. Second, WebAssembly's structured block/loop/br_if control flow maps losslessly onto a structured intermediate representation, which shaped several of the IR design decisions described in Section 3.

2. Language Design

NovaCraft is a small, C-like, statically-typed imperative language. It supports three primitive types (int, float, bool) and single-dimension arrays of each. Functions are declared with explicit parameter and return types, and the type checker enforces that every control-flow path through a function actually returns a value of the declared type. A representative program:

```
func sumArray(arr: int[], len: int) -> int {  
    let total: int = 0;  
    let i: int = 0;  
    while (i < len) {  
        total = total + arr[i];  
        i = i + 1;  
    }  
    return total;  
}
```

A deliberate and load-bearing design constraint is that **arrays only ever appear as function parameters** — there are no array literals and no local array allocation anywhere in the language. This was not an oversight; it is what makes the length-tracking convention in Section 4 tractable without a general aliasing or shape analysis: because an array can only ever enter a function as one of its own parameters, and its length is established once, by convention, as the very next parameter (e.g. arr: int[], len: int), every array reference inside a function is provably one of that function's own parameters, and its length is known

without any cross-function or whole-program analysis.

Control flow originally consisted of `if/else` and `while`. This project's most recent iteration added two further constructs, both implemented as pure syntactic sugar with zero new intermediate-representation surface area (detailed in Section 7):

- **else if** chains of arbitrary length, for readable multi-way branching.
- **for** loops with a C-style `init; condition; update` header, for the common counted-loop idiom without hand-writing a counter variable and increment statement.

The full grammar — statement forms, operator precedence, and expression grammar — is captured formally in the project's build specification document; this report focuses on the parts most relevant to the compiler's own engineering.

3. Compiler Architecture & Pipeline

The compiler is organized as a conventional multi-pass pipeline, with every stage implemented as an independent, individually testable module and independently inspectable via CLI flags (`--emit-tokens`, `--emit-ast`, `--emit-ir`, `--emit-alloc`, `--emit-wat`):

```
source (.min)
-> Lexer          tokens
-> Parser          AST          (recursive descent + precedence climbing)
-> Semantic analysis type-checked AST (scoped symbol tables)
-> IR generation   three-address IR (+ BoundsCheck insertion)
-> Constant folding
-> Dead-code elimination
-> Range analysis  bounds-check elimination
-> Common-subexpr. elimination
-> Dead-code elimination (again, to clean up after CSE)
-> Register alloc  linear scan, 4-register budget, spilling
-> Codegen         WebAssembly text (.wat) + source map
-> Runtime harness assemble (wabt) + instantiate + run
```

3.1 Lexer & Parser

The lexer is a straightforward hand-written scanner producing a flat token stream with full line/column position tracking (needed later for precise diagnostics). The parser is a recursive-descent parser for statements and top-level declarations, with a precedence-climbing algorithm for expressions so that operator precedence and associativity (logical, equality, relational, additive, multiplicative) are handled by a single compact loop rather than a cascade of precedence-level functions. Both the lexer and parser implement panic-mode error recovery: on a lexical or syntax error, the reporter records the diagnostic and the parser resynchronizes at the next statement or declaration boundary rather than aborting, so a source file with multiple errors reports all of them in one pass instead of only the first.

3.2 Semantic Analysis

A single-pass, block-scoped type checker. Function signatures are registered before any function body is checked, permitting forward references and recursion. Each block introduces a child scope chained to its parent, and every statement and expression form has an explicit type-checking rule (assignment compatibility, array-index typing, call-argument arity and type matching, operator typing rules — e.g. `%` is restricted to int operands because WebAssembly has no f32 remainder instruction). Critically, this stage also proves — statically, by structural recursion over the control-flow tree — whether a function is guaranteed to return a value on every path, rejecting it otherwise.

3.3 Intermediate Representation

The IR is a three-address code (every instruction has at most one destination and defines it from at most two source operands), but with one deliberate departure from the textbook design: `if` and `while` are kept as **structured** nodes carrying nested instruction lists (`thenBody/elseBody`, and a loop body) rather than being flattened into labels and conditional jumps. This decision was made for three concrete reasons, not just style:

- It maps losslessly onto WebAssembly's own structured block/loop/br_if control flow — the eventual codegen target — with no label/goto-to-structured control recovery algorithm (e.g. a Relooper pass) required.
- The range-analysis fixed-point iteration required for loop bounds proofs (Section 4) is naturally expressed as "re-analyze this loop's own body list until its ranges stabilize."
- Linear-scan register allocation still applies cleanly: a separate flattening pass (regalloc.ts) assigns every instruction a linear program-point index purely for computing live ranges, while every other pass (optimization, codegen) operates on the nested shape directly.

Every array access is compiled into an explicit `BoundsCheck(index, length)` instruction at IR-generation time, before any optimization runs — bounds checking is not bolted on afterward, it is the language's baseline semantics, and every later pass either proves one redundant or leaves it in place.

4. Deep Dive: Bounds-Check Elimination

This is the project's central technical contribution. A plain numeric interval analysis (tracking a $[lo, hi]$ range per register) is necessary but not sufficient: for the canonical `sumArray` example, `len` is an unconstrained function parameter with range $[-\infty, +\infty]$, so no purely numeric interval for the loop counter `i` can ever be shown to be strictly less than `len`. The analysis therefore tracks two complementary facts forward through each function simultaneously:

- **Numeric ranges.** A standard interval per register, with a fixed-point loop over each while body (re-analyzing until ranges stop changing) and a widening step — any bound still moving after two rounds is pushed to infinity, guaranteeing termination. This proves facts like “`i ≥ 0`”.
- **A symbolic “less-than fact” (`condBound`).** On entering the body of a while (`x < y`) loop, the analysis records — by register identity, not by numeric value — that `x < y` holds, and invalidates that fact the instant either register is next redefined. This is what lets the analysis prove `i < len` even though `len` itself carries no useful numeric upper bound at all.

A `BoundsCheck(index, length)` is eliminated only when **both** `index.lo ≥ 0` and either the numeric interval proves `index.hi < length.lo`, or the symbolic fact map proves `index < length`. This is a deliberately conservative rule: when the analysis is in any doubt, the check is kept. That bias is not a shortcoming to be fixed later — it is the correctness invariant the entire pass exists to preserve; a false elimination would silently reintroduce the exact memory-safety hole the language is designed to prevent.

Running the compiler with `--stats` reports this decision per function, e.g. for the canonical example:

```
Bounds checks [sumArray]: 1 inserted, 0 retained (100% eliminated)
```

and, just as importantly, for a program that indexes with a parameter genuinely unrelated to the array's own length:

```
Bounds checks [unsafeGet]: 1 inserted, 1 retained (0% eliminated)
```

Both outcomes are exercised by automated tests (Section 9); a pass that could only ever report 100% elimination would be far less convincing evidence of a real analysis than one that demonstrably knows its own limits.

5. Deep Dive: Register Allocation & Calling Convention

WebAssembly itself has no register file — it is a stack machine with an unlimited number of per-function local variables. To make register allocation's decisions actually observable in the generated code (rather than only in a printed table), the compiler performs real linear-scan register allocation (Poletto & Sarkar) with a fixed, configurable physical-register budget (`r0..r{budget-1}`, default 4) and routes every virtual-register access through the resulting allocation table.

Live ranges are computed by flattening the structurally-nested IR into a single linear sequence of program points — a separate pass from optimization and codegen, which continue to operate on the nested shape. A loop introduces a subtlety the flattening pass has to handle explicitly: a variable read near the *top* of a loop body is needed again on the next iteration, even though its last textual occurrence in a single linear pass may appear earlier than some other register's use later in the same iteration. The fix is to add one

synthetic “use” of every register touched anywhere inside a loop, placed immediately after the loop, conservatively extending each such register's live range across the loop's entire back edge.

On register-budget overflow, the allocator spills the interval with the furthest next use to an explicit spill slot in linear memory — addressed relative to that call's own stack frame, not a global offset. `--emit-alloc` makes the allocator's decisions inspectable directly:

```
Register allocation for sumArray (budget=4):
  %t0 -> r2
  arr -> r0
  i -> r3
  len -> r1
  total -> spill[0]
```

Calling convention: a mutable global `$sp` (initialized to 65536, the start of WebAssembly's second linear-memory page) acts as a downward-growing stack pointer. Each call decrements it by its own frame's spill-slot size in the prologue, captures the resulting address as a local `$frameBase`, and restores `$sp` before every return (using WebAssembly's native multi-exit return instruction directly, with no epilogue-via-branch machinery needed). Because `$sp` is a single global shared across all activations, and each call only ever touches the region between its own `frameBase` and `frameBase + frameSize`, this remains correct under recursion: nested and recursive calls simply push further down from whatever value the caller left `$sp` at.

6. Runtime Safety: Traps and Source Maps

A retained BoundsCheck that fails at runtime does not simply produce a bare WebAssembly trap; it produces the exact NovaCraft source location of the offending access. This works via a small side channel: before executing WebAssembly's unreachable instruction, the failing index, the array's length, and a small globally-unique id identifying *which* BoundsCheck fired are written to three fixed offsets in linear memory. The id is necessary because the JavaScript WebAssembly.RuntimeError the host catches carries no instruction offset of its own — with a function potentially retaining several checks, the id is the only way to know which one actually trapped. A JSON source map, generated alongside the .wat output, maps each retained check's id back to its exact source line and column, and the Node-side runtime harness reads the side channel and looks up that entry to produce the final diagnostic:

```
Runtime error: array index out of bounds at bounds_violation.min:5:12 (index=100, length=5)
```

7. Engineering Additions: Control-Flow Sugar, CSE, and a Codegen Fix

This iteration of the project extended the language and compiler along three independent axes, each validated by new automated tests before being considered complete.

7.1 for / else if as zero-cost sugar

Both constructs were deliberately implemented with no new intermediate-representation node. `else if` is rewritten during parsing into an ordinary `else` block containing a single nested `if` statement, so every later stage's existing recursive handling of nested `if` already does the right thing — no other file needed to change. `for (init; cond; update) { body }` desugars one stage later, during IR generation, directly into the same structured `while` IR node a hand-written `while` loop produces (`init` runs once in a scope that outlives the loop body; `update` is appended to the end of the body). The practical consequence: range analysis, register allocation, and code generation handle for loops completely for free — they only ever see `while`. The alternative (a first-class `ForInstr`) was explicitly rejected because it would have required duplicating every one of those passes' loop-handling logic for identical runtime behavior.

7.2 Common-subexpression elimination

A fourth optimization pass was added: within a straight-line instruction list, a binary operation that recomputes the same operator over the same still-live operands as an earlier one is rewritten into a cheap register-to-register move instead of being recomputed, with a following dead-code pass cleaning up whatever that rewrite leaves unused. Its placement in the pipeline was a deliberate correctness decision, not an afterthought: it runs *after* range analysis rather than alongside the earlier passes, because the bounds-check elimination pass proves its “less-than fact” (Section 4) by pattern-matching a specific `binop` instruction directly. Running CSE first risked silently rewriting that exact instruction into a move, defeating the pattern match and quietly weakening the bounds-check elimination — a subtle regression that would only show up as “elimination got worse”, not as a crash.

7.3 A real, latent codegen bug found and fixed

Adding else if chains made a specific program shape far more common: a function whose last statement is an if/else chain where every branch returns. Semantic analysis had always accepted this shape as a valid guaranteed-return function. Compiling it, however, produced a WebAssembly module that failed wabt's own validator with a type mismatch. The root cause: code generation emits a bare if with no declared result type whenever neither branch leaves a value on the stack (each ends in an explicit return instead) — and WebAssembly's validator has no reachability analysis of its own, so a block's type is exactly its declared immediate, regardless of whether every path inside it actually diverges. When that untyped if was the very last instruction in a function declared to return a value, validation failed even though the code could, in fact, never fall through. The fix: every function body now ends with an unconditional trailing unreachable instruction, which type-checks against any expected result and is genuinely dead code whenever a function actually returns on every path (which semantic analysis already guarantees) — closing the validator's gap for free.

8. Case Study: A Real Application and the Bug It Found

To validate the compiler against something other than its own example programs, this iteration built a genuine downstream application: a moving-average anomaly detector for numeric time series — the same category of check a real metrics or log-monitoring pipeline runs to flag a latency spike, a sensor-reading fault, or an unusual queue depth.

The NovaCraft function `movingAvgFlag` walks a data array once; for each interior index `i` it computes the mean of a symmetric window around `i`, compares `data[i]` against that local mean, and reports the index if the deviation exceeds a caller-supplied threshold. A small Node command-line tool (`tools/anomaly-cli.ts`) reads a real data file, compiles this function to WebAssembly, executes it, and prints the flagged points:

```
$ npm run anomaly -- data/sample-latency-ms.csv --window 3 --threshold 30

Bounds checks [movingAvgFlag]: 2 inserted, 1 retained (50% eliminated)

60 points, window=3, threshold=30
10 anomalies flagged:
  [10] value=48
  [11] value=54
  [12] value=210
  [13] value=195
  ...
  [30] value=4
  [45] value=260
```

This program was chosen specifically because it accesses the *same* array two different ways in one function: a direct access at index `i`, which the range analysis should be able to relate back to the enclosing loop's own bound, and a windowed access at $i \pm \text{window}$ inside a nested inner loop, which is genuinely not provable by this analysis and must remain checked. Building it did not go smoothly on the first attempt, and that failure is the more interesting result.

8.1 The bug

On first compiling `movingAvgFlag`, **both** bounds checks were retained — including the direct `data[i]` access, which was expected to be eliminated. Tracing the analysis revealed the cause was not specific to this program's exact shape but a genuine, general defect in the range-analysis pass: whenever *any* while loop finished being analyzed, its exit state unconditionally reset the **entire** symbolic “less-than fact” map to empty. This is the correct behavior for that loop's *own* condition-derived fact — once a loop exits, its own condition is false, so that specific fact no longer holds — but it also erased every *other*, completely unrelated fact that happened to be valid on entry, including facts about registers the loop's body never even touches. Here, the outer loop's own `i < len` fact was being destroyed purely because an unrelated inner windowed loop had finished, even though that inner loop only ever reads and writes its own index variable and running sum.

8.2 The fix

The exit state's fact map is now computed correctly: it keeps every fact that was valid on entry to the loop, and discards only the facts whose left- or right-hand register was actually redefined somewhere inside that specific loop's own condition or body — mirroring the same “invalidate exactly what a subtree defines” pattern already used elsewhere in the optimizer for constant folding and common-subexpression elimination. A loop's own condition-derived fact still does not survive its own exit, because the loop variable it concerns is essentially always among the registers that loop itself redefines.

The fix is externally visible in the same program's own statistics: the direct `data[i]` access is now correctly proven safe and its check eliminated, while the windowed `data[j]` access — correctly — still is not:

Access	Before fix	After fix	Correct?
<code>data[i]</code> (direct, loop-bound)	Retained	Eliminated	Yes — genuinely provable
<code>data[j]</code> (windowed, $i \pm \text{window}$)	Retained	Retained	Yes — genuinely not provable

This is arguably the strongest evidence in the whole project that the analysis is real: it was improved by a bug a genuine application exposed, and the resulting fix produces exactly the expected outcome on each of the two accesses — one proven, one honestly not — rather than either over- or under-eliminating.

The fix is covered by two regression tests: one asserts the elimination count directly against the compiler's own internal statistics, and a second — an independent JavaScript reference implementation of the identical algorithm — checks the compiled WebAssembly's actual flagged output against it on both a synthetic single-spike array and a realistic 60-point latency series, so a future regression would be caught by behavior, not just by an internal counter.

9. Testing Strategy & Continuous Integration

The test suite mirrors the pipeline architecture directly: one dedicated test file per compiler stage, plus end-to-end tests that compile a program all the way to WebAssembly and actually execute it. 34 tests currently pass across 10 suites.

Test file	What it verifies
lexer.test.ts	Correct tokenization; precise error location on an illegal character.
parser.test.ts	Well-formed ASTs for real programs; error recovery; for/else-if grammar.
semantic.test.ts	Type-mismatch and undeclared-identifier rejection; missing-return detection.
ir.test.ts	Correct BoundsCheck insertion count per function.
rangeAnalysis.test.ts	Elimination on provably-safe accesses; retention on unprovable ones; the condBound-survival regression from
cse.test.ts	A duplicate expression is actually folded to a move, without changing the result.
regalloc.test.ts	Spilling occurs under register pressure; no spilling when unnecessary.
compile.test.ts	The --no-bounds-elim comparison path used by the benchmark stays correct.
anomalyDetect.test.ts	Compiled WASM output matches an independent JS reference implementation.
e2e.test.ts	Real WebAssembly execution: sums, recursion, exact trap diagnostics, CLI smoke test.

A GitHub Actions workflow runs the TypeScript type checker, the full test suite, and a production build on every push and pull request, against both Node.js 18 and 20, so a regression is caught before it reaches the main branch rather than being discovered later.

10. Performance Evaluation

A dedicated benchmark answers the question the whole bounds-check-elimination effort exists to answer: does proving a check redundant actually make the program faster? The benchmark compiles two programs, each twice from the identical source — once normally (checks eliminated) and once with the elimination pass explicitly disabled (every check retained, running on every single array access) — assembles all four resulting modules to real WebAssembly, and times each over a 4,000,000-element array across 20 calls, after a warmup:

Scenario	Checks/iter.	Eliminated (ms/call)	Retained (ms/call)	Speedup	Overhead removed
sumArray (read)	1	12.35	16.42	1.33x	32.9%
scaleArray (read + write)	2	10.20	17.82	1.75x	74.7%

Both variants of each scenario are verified to compute the identical, correct numeric result before any timing number is trusted — the benchmark is measuring the same proven result reached with and without

paying for the check(s), not two programs that merely happen to run at different speeds. The second scenario, which performs both a read and a write through the same index each iteration, shows the benefit scaling with how many checks a hot loop actually contains: nearly double the speedup of the single-check scenario.

11. Limitations & Future Work

- **Arrays only as parameters.** There are no array literals and no local array allocation. This is what makes the current length-tracking convention tractable, but it means a NovaCraft program cannot construct an array value from within the language itself — every array a program uses must ultimately arrive from its caller (or, at the top, from the host). Supporting array literals or local allocation would require a genuine redesign of the array-length convention described in Section 2, not an incremental addition.
- **Relational reasoning is limited to direct comparisons.** The symbolic “less-than fact” is recorded only for a loop's own literal condition expression. Section 8's windowed access ($i \pm \text{window}$ against len) is a concrete example of a genuinely safe pattern (given suitable guard conditions) that the current analysis correctly declines to prove, rather than risking an unsound elimination. Extending the analysis to compose facts across simple arithmetic (as some production compilers do) is a natural next research direction.
- **No strings, structs, or multi-dimensional arrays.** The type system currently covers three primitives and single-dimension arrays of them; richer data types were out of scope for this iteration.
- **Single-length-parameter convention.** A function with two array parameters needs two separate length parameters (even if they are numerically equal at every call site) for both arrays' checks to be independently provable, since the analysis's symbolic facts are keyed by register identity, not by runtime value — a real, intentional trade-off documented and exercised directly by the anomaly-detector case study.

12. Conclusion

NovaCraft demonstrates, end to end and with real, executable output, that the tension between memory-safe array access and its runtime cost is not a fundamental trade-off but an engineering problem a real compiler can solve: prove what can be proven, keep what cannot, and measure that the result is actually faster. Every stage of the pipeline — lexing through WebAssembly code generation — is genuinely implemented, independently testable, and inspectable via the CLI. The project's most recent iteration extended the language with zero-cost control-flow sugar, added a fourth optimization pass with a carefully-reasoned position in the pipeline, fixed a real WebAssembly-validation bug the sugar exposed, and — most tellingly — built a genuine downstream application that both proved the compiler's value on a realistic workload and surfaced a real defect in its core analysis, which was diagnosed, fixed, and covered by regression tests before being considered done. That progression — write a real program, find a real bug, fix it, prove the fix — is the strongest evidence available that this is a working compiler and not a demonstration of one.

Repository layout, the full grammar, and every non-obvious design decision's rationale are recorded in the project's own README.md and DEVLOG.md.